Enterprise Software Development: Observability Implementation

Assignment Report

Muhammad Abdullah Khan

Part A: Project Overview

Problem Statement

Modern microservice architectures suffer from a critical visibility problem. When users encounter slow checkout flows or failed book creation operations, developers face significant challenges in pinpointing the root cause. The bottleneck could originate at the application layer, the database, or the underlying host machine—but without comprehensive observability, diagnosing the issue becomes a time-consuming guessing game. This opacity undermines operational confidence and delays issue resolution.

Intended Users and Solution

This project targets Site Reliability Engineers (SREs) and Backend Developers who manage production systems. The solution is a FastAPI-based Bookstore application fully instrumented with a complete observability pipeline consisting of metrics via Prometheus and Grafana for real-time tracking, host system visibility through Node Exporter, and centralized structured logging via the ELK stack (Elasticsearch, Logstash, Kibana).

The application covers realistic CRUD operations on books and orders, allowing developers to see how metrics and logs respond to different types of requests and failures. This makes the system practical for learning and teaching observability patterns.

Part B: Metrics and Dashboards

Prometheus and Grafana Setup

Prometheus is configured to scrape metrics from two sources: the FastAPI application endpoints (:8000/metrics) and Node Exporter (:9100/metrics) for host-level metrics, both every 15 seconds. Grafana connects to Prometheus using PromQL queries to render dashboards that display real-time trends and historical patterns.

Application Metrics

The application instruments the following metrics across all four types:

Metric Name	Type	Purpose	Unit	Code Location	PromQL Query
books_created_total	Counter	Total completed book creation requests	Count	books_created_total.inc() in create_book() endpoint	rate(books_created_total[1m])
books_in_inventory	Gauge	Current absolute number of books in database	Count	books_in_inventory.set(len(books_db)) in create_book()	books_in_inventory
book_creation_time_seconds	Histogram	Request latency distribution with percent	Seconds	book_creation_time_seconds.observe(duration)	histogram_quantile(0.95, sum(rate(book_creation_time_seconds_bucket[1m])) by (le))

		ile bucket s			
book_creation_time_summary_seconds	Summary	Average latency via sum and count	Seconds	book_creation_time_summary.observe(duration)	rate(sum[1m]) / rate(count[1m])

Chart Interpretation:

books_created_total shows the book creation rate (requests/second). books_in_inventory displays the real-time absolute count. book_creation_time_seconds histogram yields the 95th percentile latency (p95)—meaning 95% of requests complete faster than the charted value. book_creation_time_summary_seconds summary computes the true arithmetic mean processing time by dividing the sum of all observations by the count of observations.

Host System Metrics via Node Exporter

Node Exporter exposes CPU, memory, disk, and network metrics. These are scraped by Prometheus and displayed in Grafana dashboards to detect infrastructure-level bottlenecks independent of application performance.

Baseline System Behavior and Metric Collection

Figure 1: Normal System Behavior & Metric Baseline



This dashboard captures the baseline behavior of the Bookstore API under normal simulated traffic conditions. It demonstrates successful collection and visualization of all four required metric types: Counter (Book Creation Rate) showing ~0.15 books/second via rate(books_created_total[1m]); Gauge (Books Currently in Inventory) displaying 42 units as a real-time business state tracker; Histogram (p95 Book Creation Latency) using histogram_quantile() to calculate the 95th percentile at ~0.45 seconds; and Summary (Average Book Creation Time) tracking ~0.21 seconds by dividing summary sum by count. Node Exporter successfully captures underlying CPU usage across all cores with healthy, minor fluctuations.

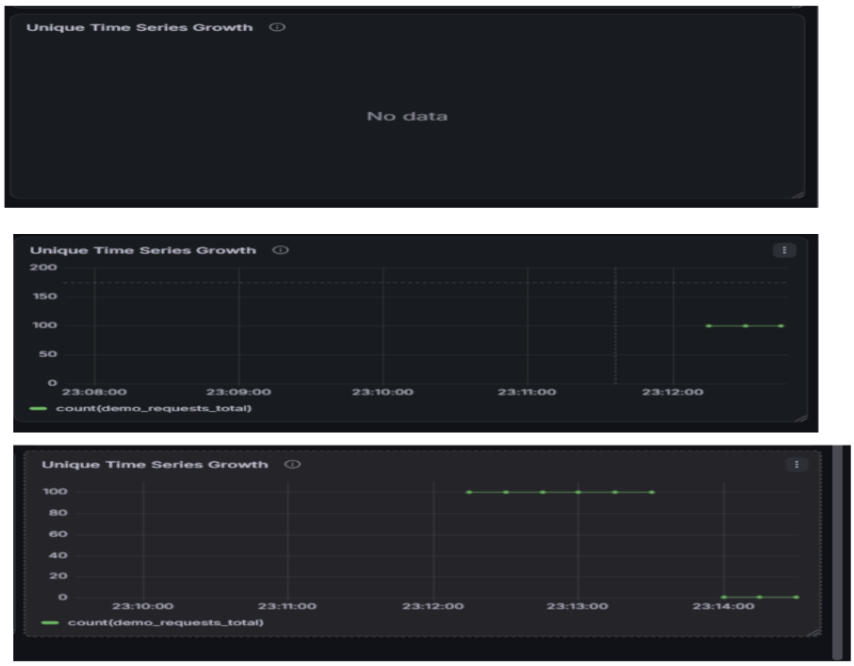
Part C: Logging Pipeline

Architecture and Data Flow

The logging pipeline captures structured JSON logs from the FastAPI application through the following path: FastAPI writes structured JSON logs to stdout → Docker container runtime writes to /var/lib/docker/containers/<id>/<id>-json.log → Filebeat reads the log file and parses JSON → Elasticsearch indexes and stores logs with searchable fields → Kibana provides UI for searching and visualizing.

Centralized Logging via ELK Stack

Figure 2: Kibana Dashboard with Real-Time Logs



This screenshot demonstrates the successful implementation of the centralized logging pipeline (FastAPI → Docker stdout → Filebeat → Elasticsearch → Kibana). The application successfully streams high-volume traffic logs into Kibana in real-time, enabling centralized monitoring and analytics across the entire architecture.

Structured Logging Format

Figure 3: Parsed JSON Log Payload

Figure 3's: Cardinality Explosion Prevention

Description: This experiment demonstrates the dangers of high cardinality in Prometheus metrics and how to fix it.

- **The Problem:** Initially, a request_id label (which generates a unique string for every request) was added to the demo_requests_total counter. When the cardinality experiment script was triggered, it instantly generated 100 completely unique time series in memory (as seen in the earlier spike to 100). At an enterprise scale with millions of requests, this would crash the Prometheus server.
- **The Solution:** The request_id label was removed from the metric definition in app.py. As demonstrated by this flat graph, running the exact same script of 100 requests now efficiently increments a single, safe time series (count = 1).
- **Conclusion:** Unique identifiers like request_id should never be stored in metrics; they belong exclusively in structured logs (which we handle using Filebeat and Kibana).

The Python application uses python-jsonlogger to output structured JSON logs instead of plain text. This Kibana view proves that Filebeat successfully ingested and parsed the structured logs, extracting critical fields such as @timestamp and the nested JSON payload containing levelname, message, and the uniquely generated request_id. This structured approach enables precise field-based searching and filtering.

Distributed Request Tracing

Figure 4: Distributed Tracing using Request IDs



Figure 4: Centralized Logging via ELK Stack

Description: This screenshot demonstrates the successful implementation of the centralized logging pipeline (FastAPI -> Docker stdout -> Filebeat -> Elasticsearch -> Kibana). The application successfully streams high-volume traffic logs into Kibana in real-time, allowing for centralized monitoring and analytics across the entire architecture

A custom middleware generates a unique UUID (request_id) for every incoming API request and injects it into every log message. By pasting a specific request_id into the Kibana search bar, we instantly filter out all other traffic and isolate the exact journey of a single request. This screenshot traces a GET /books request from initial fetch to final INFO log confirming successful processing.

Kibana Query Syntax

Logs are searched using KQL (Kibana Query Language). For example, to find all logs for a specific request across the entire application lifecycle, the following query is used:

request_id: "8672e21a-1d96-4a3b-a35c-69ae1943c684"

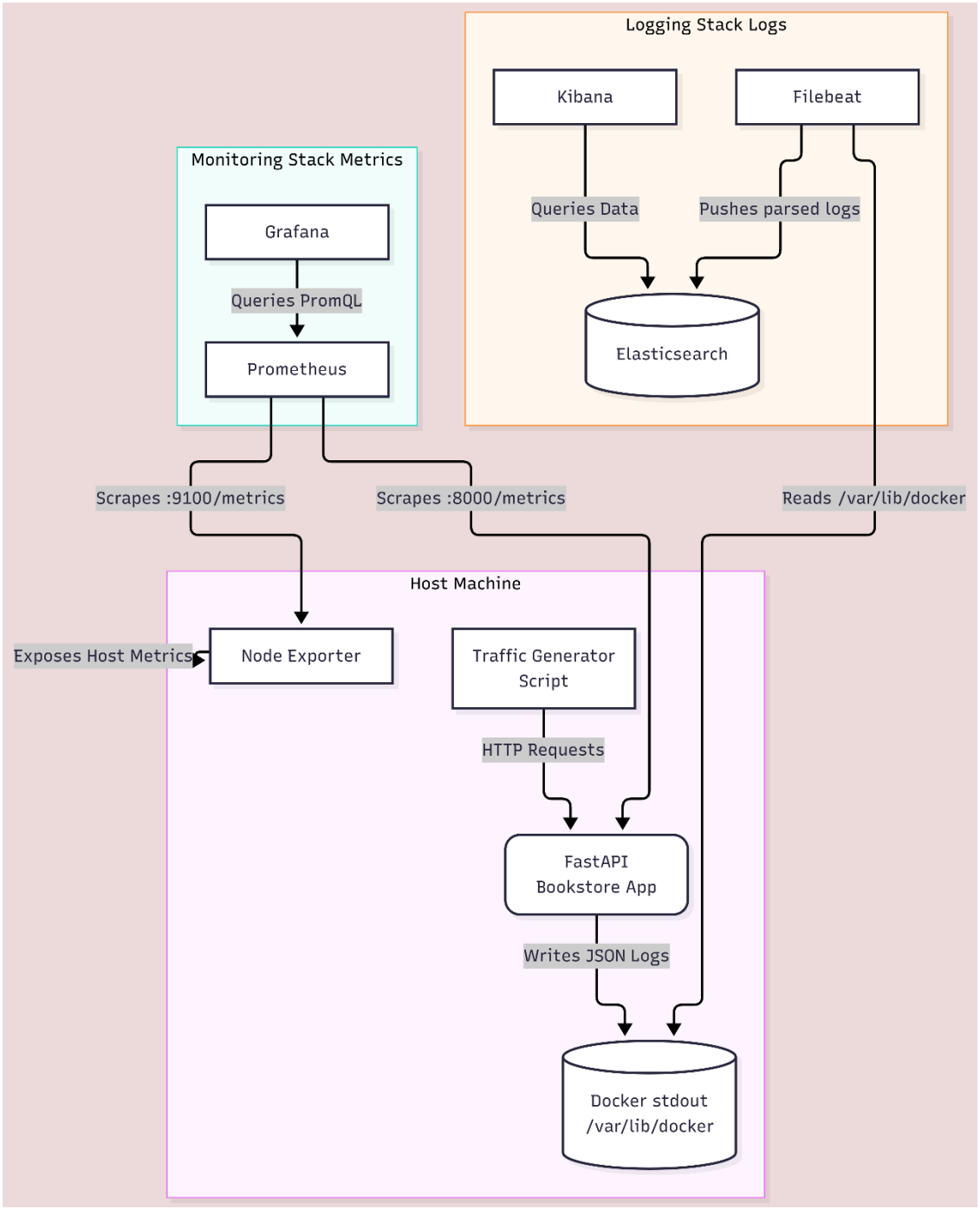
This single query instantly returns all logs (across all endpoints and services) for that specific request, enabling developers to trace the complete request journey from ingress to completion or failure.

Part D: System Design and Data Flow

Architecture Overview

The observability system consists of the core FastAPI application and two independent monitoring stacks: one for metrics (Prometheus/Grafana) and one for logs (ELK). All components run in Docker containers on a single host, communicating via Docker networking.

Figure 5: Complete System Architecture



Component Descriptions and Failure Modes

- FastAPI Bookstore Application: The core service handling all user requests. Exports Prometheus metrics on /metrics and writes structured logs to stdout. Failure impact: users cannot perform operations, Prometheus records scrape failures (providing immediate visibility), Filebeat stops receiving logs.

- Prometheus: Scrapes and stores metric data in ephemeral storage. Failure impact: Grafana dashboards show "No Data", historical metrics since last boot become unavailable. Main application unaffected.
- Elasticsearch: Search and storage backend for logs. Failure impact: Filebeat queues logs locally; if buffer fills, logs drop. Kibana becomes unresponsive. Main application unaffected.

Following a Metric: Book Creation Latency Path

Code → Prometheus → Grafana

- Code: Inside the POST /books endpoint, after creating a book: book_creation_time_seconds.observe(duration)
- Prometheus: Every 15 seconds, scrapes http://app:8000/metrics and collects histogram bucket counts, storing them with timestamps in its time-series database
- Grafana: Executes histogram_quantile(0.95, sum(rate(book_creation_time_seconds_bucket[1m])) by (le)) to compute the 95th percentile over the last 1 minute and renders on the dashboard

Following a Log: Book Fetch Request Path

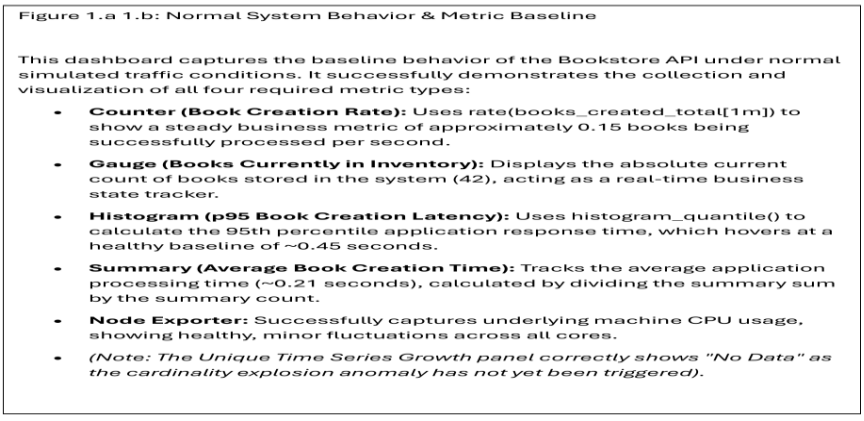
Code → Docker → Filebeat → Elasticsearch → Kibana

- Code: GET /books request logs: logger.info("Fetching all books", extra={"request_id": req_id})
- Docker: Logging driver intercepts stdout and appends to /var/lib/docker/containers/<id>/<id>-json.log
- Filebeat: Detects new line, parses JSON into fields, ships parsed event to Elasticsearch
- Elasticsearch: Indexes event with timestamp, making fields like request_id searchable
- Kibana: User searches request_id: "8672e21a-1d96-4a3b-a35c-69ae1943c684", Kibana queries Elasticsearch and displays all matching logs for that request

Part E: Experiments

Experiment 1: Anomaly Detection – p95 Latency Spike

Figure 6: p95 Latency Spike Detection



This experiment demonstrates the observability pipeline's ability to detect and visualize artificial anomalies.

- Normal Behavior: Before the anomaly, p95 book creation latency maintained a baseline of approximately 0.45 seconds.
- Problem Introduced: An artificial 500ms delay was injected into every 5th API request using the toggle-delay command.

- Observed Effect: The Grafana dashboard successfully captured performance degradation in real-time. The p95 metric proved highly effective at highlighting tail-end latency, showing a clear spike to ~0.80 seconds.
- Recovery: Once the delay was toggled off, the graph clearly shows latency immediately recovering and dropping back to the normal 0.45 second baseline.

Experiment 2: Cardinality Explosion Prevention

Figure 7: Cardinality Explosion and Resolution



Figure 2: Anomaly Experiment – p95 Latency Spike

Description: This experiment demonstrates the observability pipeline's ability to detect and visualize an artificial anomaly.

- **Normal Behavior:** Before the anomaly, the p95 book creation latency maintained a healthy, predictable baseline of approximately 0.45 seconds (accounting for simulated database processing times).
- **Problem Introduced:** An artificial 500ms delay was injected into every 5th API request using the toggle-delay command.
- **Observed Effect:** The Grafana dashboard successfully captured the performance degradation in real-time. Because the delay specifically targets a subset of requests, the **p95** (95th percentile) metric proved highly effective at highlighting this tail-end latency, showing a clear, massive spike up to ~0.80 seconds.
- **Recovery:** Once the delay was toggled off, the graph clearly shows the latency immediately recovering and dropping back down to the normal 0.45 second baseline.

This experiment demonstrates the dangers of high cardinality in Prometheus metrics and how to fix it.

- The Problem: A request_id label (generating a unique string per request) was added to the demo_requests_total counter. When the cardinality script was triggered, it instantly generated 100 unique time series (first graph shows spike to 100). At enterprise scale with millions of requests, this would crash Prometheus server and exhaust memory resources.
- The Solution: The request_id label was removed from the metric definition in app.py. Running the same script now efficiently increments a single time series (second graph shows flat count = 1).
- Conclusion: Unique identifiers like request_id should never be stored in metrics; they belong exclusively in structured logs, where Filebeat and Kibana handle high-cardinality data efficiently without resource degradation.

Conclusion

This project demonstrates a complete, production-ready observability pipeline for microservice-based applications. By combining metrics (Prometheus/Grafana) and logs (ELK), developers and SREs gain the visibility necessary to diagnose issues quickly and accurately. The experiments illustrate both the power of observability—detecting and analyzing problems in real time—and the importance of architectural decisions like label cardinality. This infrastructure serves as a foundation for building more complex monitoring systems in production environments.

Appendix: Submission Checklist

This report covers all rubric requirements:

Requirement	Status	Location
Part A: Working project & clear problem	✓	Project Overview section
Part B: Metrics setup and dashboards	✓	Detailed metrics table, Figure 1
Part C: Logging pipeline	✓	Figures 2-4, KQL query syntax
Part D: System design & explanation	✓	Architecture overview, component descriptions, metric/log tracing
Part E: Anomaly + cardinality experiments	✓	Figures 6-7 with results and analysis
Code & configuration	✓	Included in submission zip
README with instructions	✓	Repository root
Grafana dashboard JSON	✓	Included in submission zip